



Arquiteturas de Software

Fundamentos, estilos e padrões arquiteturais

Professor Vinícius Breda

O papel da arquitetura de software

Decisões estruturais que conectam negócio, tecnologia, qualidade e evolução.

A arquitetura de software define como um sistema será estruturado, organizado e evoluído ao longo do tempo. Ela estabelece os principais componentes da aplicação, suas responsabilidades, a forma como eles se comunicam e quais tecnologias e padrões serão utilizados.

O papel da arquitetura não é apenas “desenhar caixas e setas”. Ela serve para orientar decisões técnicas importantes e garantir que o sistema consiga atender requisitos como desempenho, segurança, escalabilidade, disponibilidade, manutenção e integração.

Uma boa arquitetura também ajuda a reduzir acoplamento entre componentes, facilitar testes, permitir mudanças com menor impacto e tornar o desenvolvimento mais previsível para o time. Em sistemas maiores, ela também influencia diretamente a forma como as equipes trabalham, como as aplicações são implantadas e como os serviços escalam.

A arquitetura busca equilibrar três pontos:

● Necessidades do negócio

O sistema precisa resolver o problema corretamente.

● Qualidade técnica

Deve ser seguro, performático, escalável e sustentável.

● Evolução

Precisa permitir mudanças futuras sem que cada alteração se torne cara ou arriscada.

Por isso, não existe uma arquitetura universalmente melhor. Clean Architecture, Hexagonal, Microserviços, Event-Driven, Serverless, REST, GraphQL e outras abordagens são ferramentas. O arquiteto precisa escolher quais fazem sentido de acordo com o contexto, tamanho do sistema, volume de usuários, equipe, infraestrutura e necessidades do negócio.

“ **Arquitetura de software é o conjunto de decisões que define como um sistema será construído hoje sem impedir que ele evolua amanhã.** ”

Clean Architecture

Organização do sistema com foco em separação de responsabilidades e desacoplamento

A Clean Architecture separa o sistema em camadas para que as regras de negócio permaneçam independentes de frameworks, bancos de dados, interface de usuário e detalhes externos.



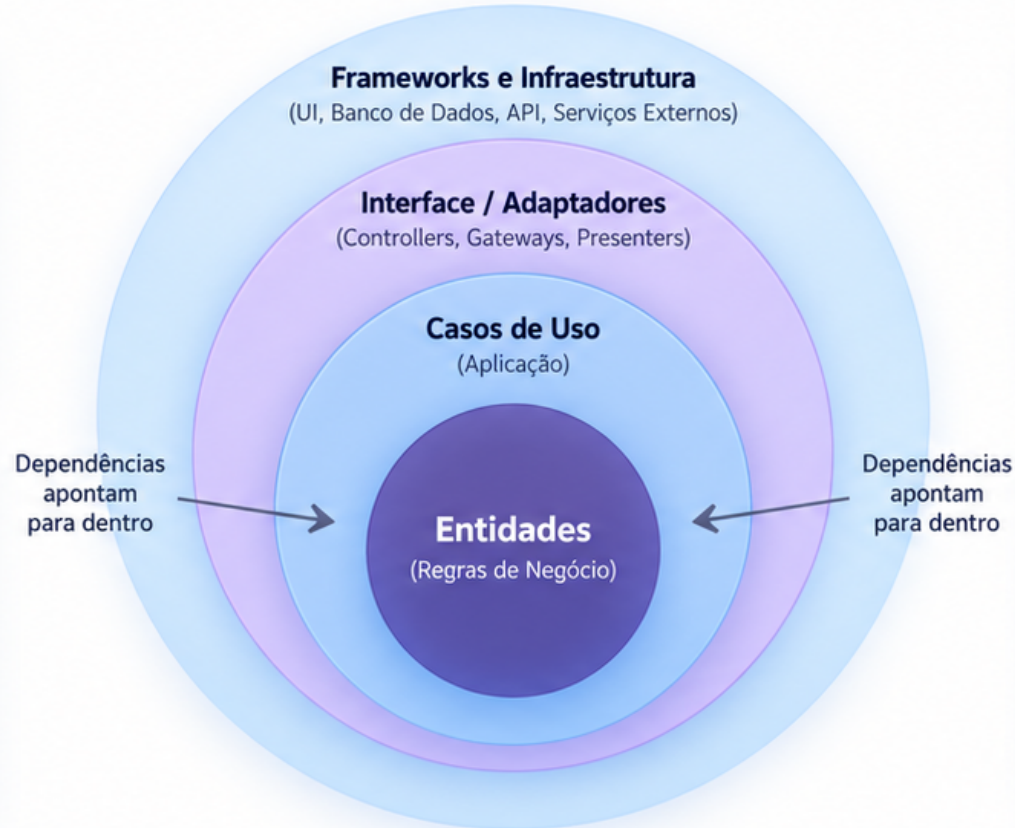
Objetivo

Proteger as regras de negócio e melhorar a manutenibilidade do sistema.



Vantagens

- baixo acoplamento
- maior testabilidade
- facilidade de manutenção
- evolução mais segura.



Quando usar

Útil em sistemas que tendem a crescer e que exigem manutenibilidade a longo prazo.



Exemplo prático

API, banco de dados e interface web dependem do núcleo da aplicação (regras de negócio), não o contrário.



Ideia principal: o núcleo do sistema não deve depender da infraestrutura.

Arquitetura Hexagonal

Portas e adaptadores para proteger o domínio da aplicação

O núcleo de negócio permanece isolado, enquanto as portas definem contratos de entrada e saída e os adaptadores conectam tecnologias externas, como bancos de dados, interfaces e serviços.



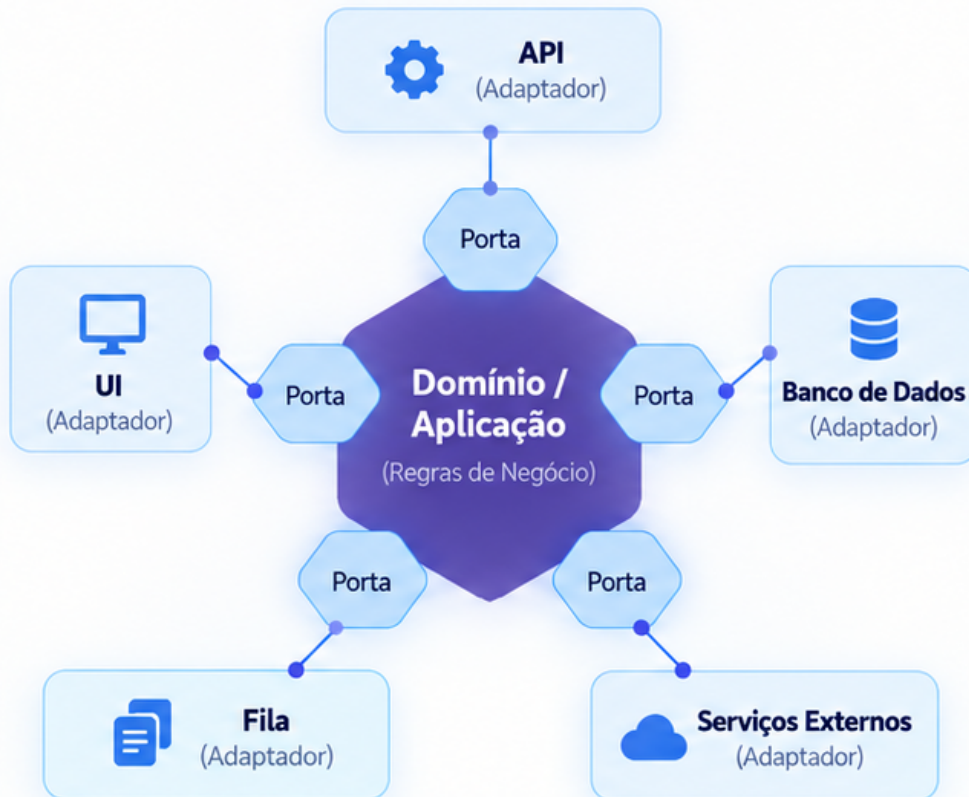
Objetivo

Isolar o domínio das dependências externas.



Vantagens

- desacoplamento
- facilidade para testar
- troca de tecnologias
- maior flexibilidade.



Quando usar

Útil quando o núcleo de negócio deve permanecer protegido dos detalhes de infraestrutura.



Exemplo prático

Uma regra de negócio pode ser utilizada pela API, interface web e worker em segundo plano através de diferentes adaptadores.



Ideia principal: o domínio conhece contratos, não tecnologias externas.

Arquitetura em Camadas

Separação do sistema em níveis de responsabilidade

A aplicação é organizada em camadas para separar a apresentação, as regras de negócio e as preocupações técnicas, tornando o sistema mais organizado, manutenível e evolutivo.



Objetivo

Organizar as responsabilidades de forma clara, separando as preocupações do sistema.



Vantagens

- estrutura simples
- facilidade de entendimento
- manutenção organizada
- boa divisão de responsabilidades.



Apresentação

(Interface do Usuário)

Responsável por exibir as informações e receber as entradas do usuário.



Aplicação

(Casos de Uso / Serviços)

Orquestra o fluxo da aplicação, executa regras de negócio e coordena as operações.



Domínio

(Regras de Negócio)

Contém as entidades, regras de negócio e a lógica do sistema.



Infraestrutura

(Persistência / Serviços Externos)

Responsável pelo acesso a dados, integrações e serviços técnicos (banco de dados, APIs, etc.).



Quando usar

Útil em aplicações empresariais tradicionais e em sistemas com uma estrutura mais direta e bem definida.



Exemplo prático

Uma tela web chama a camada de aplicação, que executa as regras de negócio e acessa a persistência por meio da camada de infraestrutura.



Ideia principal: cada camada possui um papel bem definido.

Model, View e Controller organizando interface, dados e fluxo

O MVC separa a interface de usuário, a representação de dados e a lógica de fluxo de requisições em componentes distintos, permitindo um sistema mais organizado, flexível e fácil de manter.



Objetivo

Separar a camada visual da lógica de aplicação e de fluxo.



Vantagens

- organização da interface
- reuso
- separação de responsabilidades
- facilidade de manutenção.

Ações do usuário
(requisições)



Controller

Recebe as ações, atualiza o Model e seleciona a View.

Atualiza
o Model

Seleciona
a View



Model

Representa os dados e as regras de negócio.

Dados
(para exibição)



View

Apresenta os dados ao usuário.



Quando usar

Útil em aplicações web e em sistemas com interação baseada em interface.



Exemplo prático

Um formulário envia uma requisição para o Controller, que atualiza o Model e retorna uma View.



Ideia principal: a interface não deve concentrar toda a lógica.

CÓDIGO MAIS LIMPO
APLICAÇÕES MAIS EVOLUTIVAS

Arquitetura Monolítica

Toda a aplicação funcionando como um único sistema

Na arquitetura monolítica, todos os módulos, funcionalidades e componentes são empacotados e implantados juntos em uma única aplicação.



Objetivo

Simplificar a estrutura inicial e o processo de implantação.



Vantagens

- simplicidade inicial
- implantação única
- facilidade para começar
- menos complexidade operacional



Quando usar

Útil em projetos menores, MVPs ou equipes pequenas.



Exemplo prático

Uma única aplicação web responsável por todos os domínios da aplicação, implantada como uma única unidade.



Ideia principal: tudo evolui junto dentro do mesmo sistema.

APLICAÇÕES HOJE
GRANDES CONQUISTAS AMANHÃ

Microserviços

Serviços menores, independentes e especializados

O sistema é dividido em serviços autônomos que podem ser desenvolvidos, implantados e escalados de forma independente.



Objetivo

Dividir o sistema em capacidades de negócio autônomas.



Usuários

Gerencia contas, perfis e permissões.



Banco de Dados (Usuários)



Pedidos

Gerencia pedidos, itens e histórico.



Banco de Dados (Pedidos)



APIs /
Mensageria



Pagamentos

Processa pagamentos e transações.



Banco de Dados (Pagamentos)



Notificações

Envia e gerencia comunicações.



Banco de Dados (Notificações)



Quando usar

Útil em sistemas maiores, com múltiplos times e diferentes necessidades de escalabilidade.



Vantagens

- escalabilidade por serviço
- deploy independente
- autonomia de times
- evolução isolada.



Exemplo prático

O serviço de pagamentos pode escalar independentemente sem afetar os serviços de usuários ou catálogo.



Ideia principal: cada serviço resolve uma responsabilidade específica.

Arquitetura Event-Driven

Comunicação orientada a eventos e processamento assíncrono

Na arquitetura event-driven, os componentes publicam e consomem eventos em vez de dependerem diretamente uns dos outros, permitindo uma comunicação assíncrona e mais flexível.



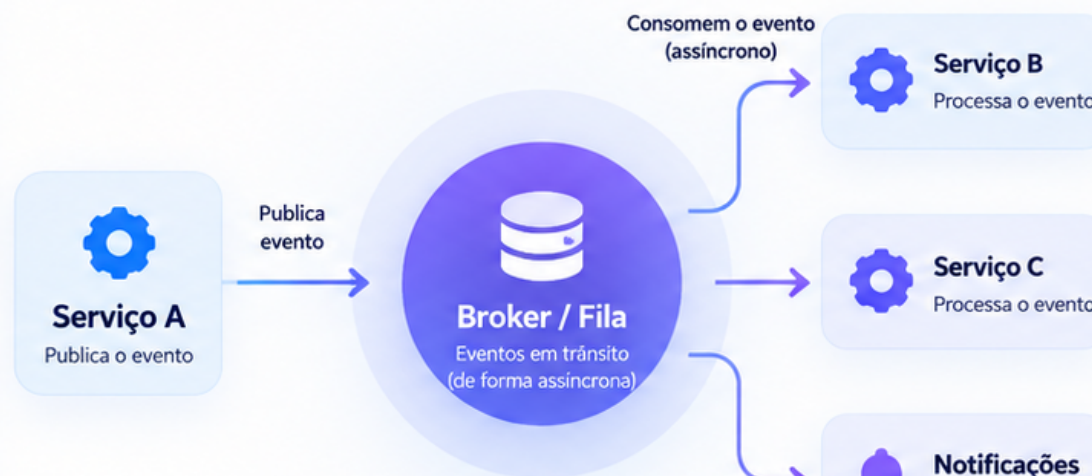
Objetivo

Desacoplar os serviços e habilitar o processamento reativo de eventos.



Vantagens

- assíncrono
- desacoplamento
- escalabilidade
- integração flexível.



Quando usar

Útil em cenários de notificações, integrações, filas e processamento distribuído.



Exemplo prático

Após a criação de um pedido, um evento dispara a cobrança, o envio de e-mail e a atualização de estoque, de forma independente.



Ideia principal: produtores e consumidores não precisam se conhecer diretamente.

Serverless

Execução sob demanda com foco em funções e serviços gerenciados

Seu código é executado sob demanda e o provedor de nuvem gerencia toda a infraestrutura, o escalonamento e grande parte da operação, permitindo que você foque na lógica de negócio.



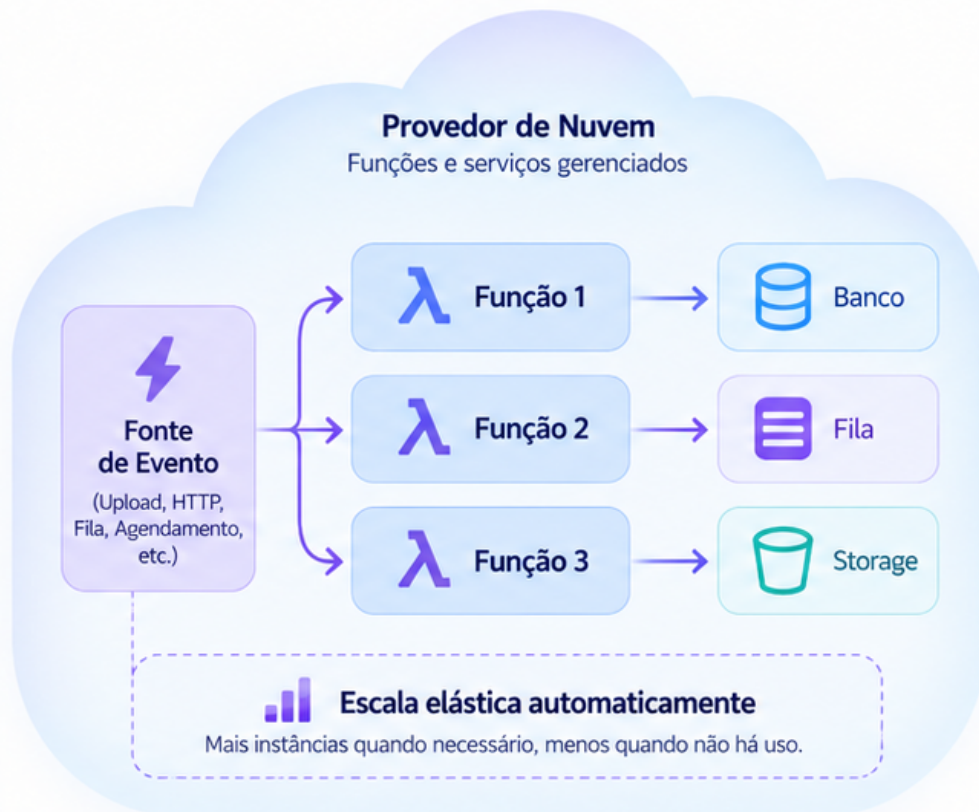
Objetivo

Executar código apenas quando necessário.



Vantagens

- cobrança por uso
- escala automática
- menos operação
- agilidade



Quando usar

Útil em automações, cargas de trabalho variáveis e processamento de eventos.



Exemplo prático

O envio de um arquivo aciona uma função que valida, transforma e armazena os dados.



Ideia principal: foco na lógica de negócio, não na gestão de servidores.

REST

APIs baseadas em recursos e operações HTTP

REST expõe recursos do sistema por meio de endpoints, utilizando os verbos padrão do protocolo HTTP para realizar operações de forma simples, stateless e interoperável.



Objetivo

Expor recursos do sistema de forma simples e interoperável.



Vantagens

- simplicidade
- ampla adoção
- integração fácil
- uso de padrões HTTP



Cliente

Web, Mobile
ou outro sistema

GET /usuarios

POST /pedidos

PUT /produtos/{id}

DELETE /itens/{id}



API

Endpoints REST

/usuarios
/pedidos
/produtos
/itens
...



Banco de Dados

Armazena
e gerencia
os dados



Quando usar

Útil para APIs tradicionais, integração com aplicações web e mobile e quando há necessidade de ampla compatibilidade.



Exemplo prático

Uma aplicação lista produtos com GET e cria pedidos com POST.



Ideia principal: recursos bem definidos e comunicação simples via HTTP.

GraphQL

MAIS FLEXIBILIDADE
MAIS EFICIÊNCIA
MAIS INTEGRAÇÃO
MAIS POSSIBILIDADES

Consultas flexíveis para obter exatamente os dados necessários

GraphQL permite que o cliente solicite apenas os campos de que precisa, por meio de um único endpoint de consulta, recebendo exatamente os dados necessários.



Objetivo

Dar aos clientes flexibilidade para moldar os dados retornados de acordo com suas necessidades.



Vantagens

- menos overfetching
- respostas sob medida
- único endpoint
- boa experiência para múltiplos clientes.

Consulta

O cliente solicita apenas o que precisa.

```
query {  
  usuario(id: 1) {  
    nome  
    pedidos(limit: 3) {  
      id  
      produtos {  
        titulo  
      }  
    }  
  }  
}
```



GraphQL API

Um único endpoint que acessa múltiplas fontes de dados.



Usuários
(Dados de perfil)



Pedidos
(Histórico de compras)



Produtos
(Catálogo)

Resposta sob medida

O cliente recebe exatamente os dados solicitados.

```
{  
  "usuario": {  
    "nome": "Ana Silva",  
    "pedidos": [  
      {  
        "id": "101",  
        "produtos": [  
          { "titulo": "Notebook" }  
        ]  
      },  
      {  
        "id": "102",  
        "produtos": [  
          { "titulo": "Mouse" }  
        ]  
      },  
      {  
        "id": "103",  
        "produtos": [  
          { "titulo": "Teclado" }  
        ]  
      }  
    ]  
  }  
}
```



Quando usar

Útil quando aplicações web, mobile e outros clientes precisam de diferentes formas de dados.



Exemplo prático

Um aplicativo mobile solicita o nome do usuário, os últimos 3 pedidos e os títulos dos produtos em uma única consulta.



Ideia principal: o cliente define a estrutura dos dados que deseja receber.

APIs MAIS INTELIGENTES
MAIS VALOR PARA PESSOAS